Tribhuvan University

Faculty of Humanities & Social Sciences

OFFICE OF THE DEAN

Operating Systems

BCA — Semester 4 — 2020 — Answer Sheet

Subject Code: CACS251

Group B

Attempt any SIX questions. [6×5=30]

1. What is an Operating System? Why is it known as the resource manager and the extended machine?

[2+3]

Answer: An **Operating System (OS)** is system software that manages computer hardware and software resources and provides common services for computer programs. It acts as an intermediary between users and computer hardware. **Resource Manager:** The OS manages all hardware resources including CPU (through scheduling), memory (through allocation and deallocation), disk (through file systems), and I/O devices (through device drivers). It ensures efficient and fair resource sharing among multiple processes and users. **Extended Machine:** The OS hides the complexity of hardware from users by providing a clean, abstract interface. Instead of dealing with low-level hardware details (disk sectors, memory addresses, I/O ports), users interact with high-level abstractions like files, folders, and applications. The OS extends the bare hardware into a more convenient and powerful virtual machine.

2. What is Kernel? What role does it play in an Operating System? Differentiate between monolithic and micro kernel.

[1+1+3]

Answer: The **Kernel** is the core component of an operating system. It is the first program loaded on startup and remains in memory throughout system operation. It has complete control over everything in the system. **Role of Kernel:** • Process management: creating, scheduling, and terminating processes • Memory management: allocating and deallocating memory space • Device management: managing device communication via drivers • System calls: handling requests from applications • Security and access control **Monolithic vs Micro Kernel: AspectMonolithic KernelMicro Kernel** ArchitectureAll OS services (file system, device drivers, scheduler) run in kernel spaceOnly essential services (memory, scheduling, IPC) run in kernel space SizeLarge and complexSmall and minimal PerformanceFaster (less context switching)Slower (more message passing) ExtensibilityHarder to extend/modifyEasier to extend services ReliabilityOne bug can crash entire systemServices can fail independently ExamplesLinux, Windows (hybrid)Minix, QNX, L4

3. Suppose that a disk has 200 cylinders (0-200). The head pointer is currently at 50 and previously at 35. The queue of pending request, in FIFO order is 60, 190, 40, 125, 15, and 150. The time required to move per cylinder is 6 msec. Starting from the current head position, what is the total seek time needed to satisfy all the pending request for each of the following disk scheduling algorithms? a) FCFS b) SCAN

[2.5+2.5]

Answer: Given: • Current head position: 50 • Previous head position: 35 (so head is moving upward) • Request queue: 60, 190, 40, 125, 15, 150 • Time per cylinder: 6 msec **a) FCFS (First Come First Serve):** Order of service: 50 → 60 → 190 → 40 → 125 → 15 → 150 Seek distances: |60-50| = 10 |190-60| = 130 |40-190| = 150 |125-40| = 85 |15-125| = 110 |150-15| = 135 Total seek distance = 10 + 130 + 150 + 85 + 110 + 135 = **620 cylinders** Total seek time = 620 × 6 = **3,720 msec (3.72 seconds) b) SCAN (Elevator Algorithm):** Since previous position was 35 and current is 50, head is moving upward (toward higher cylinders). Order: 50 → 60 → 125 → 150 → 190 → 200 (end) → 40 → 15 Seek distances: |60-50| = 10 |125-60| = 65 |150-125| = 25 |190-150| = 40 |200-190| = 10 |40-200| = 160 |15-40| = 25 Total seek distance = 10 + 65 + 25 + 40 + 10 + 160 + 25 = **335 cylinders** Total seek time = 335 × 6 = **2,010 msec (2.01 seconds)**

4. Define file and directory. Explain the concept of Access Control List (ACL) and Access Control Matrix (ACM).

[2+3]

Answer: File: A file is a named collection of related information stored on secondary storage. It is the basic unit of storage in a file system and can contain data, programs, or text. **Directory:** A directory is a special file that contains a list of files and subdirectories. It provides a logical organization structure for the file system, allowing users to group related files together. **Access Control List (ACL):** An ACL is a list of permissions attached to an object (file/directory). Each entry in the list specifies which user or group is granted or denied specific access rights (read, write, execute). ACLs provide fine-grained access control. Example: File 'grades.txt' might have: • Owner: read, write • Group 'teachers': read, write • Group 'students': read only • User 'admin': full control **Access Control Matrix (ACM):** An ACM is a two-dimensional matrix where rows represent subjects (users/processes) and columns represent objects (files/resources). Each cell contains the access rights that the subject has for that object. Example matrix: File AFile BPrinter User1RWRW User2RRW- AdminRWXRWXW **Comparison:** ACL is more space-efficient for systems with many objects but few permissions per object. ACM provides a complete view but wastes space when most entries are empty.

5. What is biometric password in authentication? Explain the various system threats.

[2+3]

Answer: Biometric Authentication: Biometric authentication uses unique physical or behavioral characteristics of an individual for identification. Unlike passwords, biometric traits cannot be easily stolen, shared, or forgotten. **Types of biometric authentication: • Physical:** Fingerprint, iris/retina scan, facial recognition, palm geometry, DNA • **Behavioral:** Voice recognition, handwriting/signature dynamics, gait analysis, keystroke dynamics **Advantages:** High security, convenience (no password to remember), non-transferable **Disadvantages:** Expensive hardware, privacy concerns, false accept/reject rates, cannot be changed if compromised **System Threats:** 1. **Malware:** Viruses, worms, trojans, ransomware that damage or steal data 2. **Denial of Service (DoS):** Overwhelming system resources to make services unavailable 3. **Man-in-the-Middle (MitM):** Intercepting communication between two parties 4. **Phishing:** Fraudulent attempts to obtain sensitive information by disguising as trustworthy 5. **Privilege Escalation:** Exploiting bugs to gain higher-level access 6. **Spoofing:** Faking identity (IP spoofing, email spoofing) 7. **Buffer Overflow:** Overwriting memory to execute malicious code 8. **Insider Threats:** Malicious actions by authorized users

6. How distributed operating system is more applicable than centralized operating system? Explain the major goals of distributed operating system.

[2+3]

Answer: Distributed OS vs Centralized OS: A **centralized OS** manages a single computer with all resources in one location. A **distributed OS** manages a collection of independent computers that appear to users as a single coherent system. **Why Distributed OS is more applicable: • Resource Sharing:** Users can access remote resources (printers, files, databases) transparently • **Reliability:** If one node fails, others continue operating (fault tolerance) • **Scalability:** System can grow by adding more nodes • **Performance:** Load balancing across multiple processors improves throughput • **Communication:** Enables collaboration across geographic locations • **Cost-effective:** Network of PCs can be cheaper than a single supercomputer **Major Goals of Distributed OS:** 1. **Transparency:** Hide distribution from users (access, location, migration, replication, concurrency, failure transparency) 2. **Reliability:** System should continue functioning despite component failures 3. **Performance:** Response time should be comparable to centralized systems 4. **Scalability:** System should handle growth in users, resources, and geographic spread 5. **Openness:** System should conform to standards for interoperability 6. **Security:** Protect data and resources across the distributed environment

7. Write short notes on any TWO: a) Producer Consumer problem b) Coalescing and Compaction c) Ubuntu

[2.5+2.5]

Answer: a) Producer-Consumer Problem: The producer-consumer problem is a classic synchronization problem where one or more producer threads generate data and put it into a shared buffer, while one or more consumer threads remove and process data from the buffer. **Challenges:** • Producer must not add data when buffer is full • Consumer must not remove data when buffer is empty • Mutual exclusion is needed to prevent race conditions **Solution using semaphores:** • empty = N (buffer size) • full = 0 • mutex = 1 Producer: wait(empty); wait(mutex); add item; signal(mutex); signal(full) Consumer: wait(full); wait(mutex); remove item; signal(mutex); signal(empty) **b) Coalescing and Compaction:** These are memory management techniques to reduce external fragmentation. **Coalescing:** When a block of memory is freed, the system checks if adjacent blocks are also free. If so, they are merged into a single larger free block. This reduces fragmentation by combining small holes into larger ones. **Compaction:** All allocated memory blocks are moved to one end of memory, leaving one large contiguous free block at the other end. This eliminates external fragmentation entirely but requires dynamic relocation support and is time-consuming. **c) Ubuntu:** Ubuntu is a popular Linux-based open-source operating system developed by Canonical Ltd. It is based on Debian and follows a regular release cycle (every 6 months) with LTS (Long Term Support) releases every 2 years supported for 5 years. **Features:** User-friendly GUI, extensive software repository, strong community support, security updates, suitable for desktops, servers, and cloud computing. Uses APT package management and GNOME desktop environment by default.

Group C

Attempt any TWO questions. [2×10=20]

8. What is CPU scheduling? Write down the criteria for CPU scheduling? Consider the following set of processes, with the length of the CPU burst given in milliseconds, draw Gantt chart illustrating their execution and calculate average waiting time and turnaround time using: a) First Come First Serve b) Shortest Job First c) Non-preemptive priority (smaller number implies higher priority) d) Round Robin (quantum=1). The processes have arrived in the order P0, P1, P2, P3, P4 all at time 0.

[1+1+2+2+2+2]

Answer: CPU Scheduling: CPU scheduling is the process of selecting which process in the ready queue should be allocated the CPU. It is a fundamental OS function that improves system performance by maximizing CPU utilization and throughput while minimizing waiting time and response time. **Criteria for CPU Scheduling: • CPU Utilization:** Keep CPU as busy as possible • **Throughput:** Number of processes completed per unit time • **Turnaround Time:** Time from submission to completion • **Waiting Time:** Time spent waiting in ready queue • **Response Time:** Time from request submission to first response • **Fairness:** Each process gets fair share of CPU **Note:** The specific process burst times table was not fully visible in the original paper. A typical example would be: ProcessBurst TimePriority P0103 P111 P224 P315 P452 **a) FCFS:** P0→P1→P2→P3→P4 Gantt: | P0 (0-10) | P1 (10-11) | P2 (11-13) | P3 (13-14) | P4 (14-19) | Waiting times: P0=0, P1=10, P2=11, P3=13, P4=14 Average Waiting Time = (0+10+11+13+14)/5 = **9.6 ms** Turnaround times: P0=10, P1=11, P2=13, P3=14, P4=19 Average Turnaround Time = (10+11+13+14+19)/5 = **13.4 ms b) SJF (Non-preemptive):** P1→P3→P2→P4→P0 (shortest burst first) Gantt: | P1 (0-1) | P3 (1-2) | P2 (2-4) | P4 (4-9) | P0 (9-19) | Waiting times: P0=9, P1=0, P2=2, P3=1, P4=4 Average WT = (9+0+2+1+4)/5 = **3.2 ms** Average TAT = (19+1+4+2+9)/5 = **7.0 ms c) Priority (Non-preemptive):** P1→P4→P0→P2→P3 (lower number = higher priority) Gantt: | P1 (0-1) | P4 (1-6) | P0 (6-16) | P2 (16-18) | P3 (18-19) | Average WT = **5.4 ms**, Average TAT = **9.2 ms d) Round Robin (quantum=1):** Gantt: P0,P1,P2,P3,P4,P0,P2,P4,P0,P4,P0,P4,P0,P0,P0,P0,P0,P0,P0 (0-19) Average WT ≈ **6.8 ms**, Average TAT ≈ **10.6 ms**

■ *Diagram Required:* Gantt charts should be drawn for each scheduling algorithm showing process execution timeline.

9. Define the terms: page fault and thrashing. Consider the following page reference string: 1, 3, 5, 1, 7, 1, 5, 5, 1, 4, 3, 7, 6, 3, 4, 1. How many page faults would occur for each of the following page replacement algorithms assuming 3 page frames? a) FIFO page replacement b) LRU page replacement c) Optimal page replacement

[2+2+2+2+2]

Answer: Page Fault: A page fault occurs when a running program accesses a memory page that is mapped into its virtual address space but is not currently loaded in physical memory (RAM). The OS must then load the required page from disk into memory, which is a costly operation. **Thrashing:** Thrashing occurs when a computer's virtual memory resources are overused, leading to a constant state of paging and page faults. The system spends more time swapping pages in and out than executing actual processes. Causes: insufficient physical memory, too many processes running, poor locality of reference. **Reference String:** 1, 3, 5, 1, 7, 1, 5, 5, 1, 4, 3, 7, 6, 3, 4, 1 **Page Frames:** 3 **a) FIFO (First In First Out):** Ref1351715514376341 F11*111111114*446*666 F2-3*3333333337*7777 F3--5*55555555553*31* Page faults marked with *. Total page faults with FIFO = **9 b) LRU (Least Recently Used):** Ref1351715514376341 F11*1111111113*36*661* F2-3*33333334*444333 F3--5*57*7777777774*4 Total page faults with LRU = **7 c) Optimal Page Replacement:** The optimal algorithm replaces the page that will not be used for the longest period in the future. Ref1351715514376341 F11*111111111116*666 F2-3*33333333333333 F3--5*57*77774*47*774*1* Total page faults with Optimal = **7**

10. Define deadlock. List out the conditions that can result in resource deadlock. Explain different deadlock handling methods in details.

[2+1+7]

Answer: Deadlock: A deadlock is a situation in which two or more processes are unable to proceed because each is waiting for one of the others to release a resource. It is a permanent blocking of processes. **Four Necessary Conditions for Deadlock (Coffman Conditions):** 1. **Mutual Exclusion:** At least one resource must be non-sharable 2. **Hold and Wait:** A process holding resources can request additional resources 3. **No Preemption:** Resources cannot be forcibly taken from a process 4. **Circular Wait:** A circular chain of processes exists where each waits for a resource held by the next **Deadlock Handling Methods: 1. Deadlock Prevention:** Ensure at least one Coffman condition never holds. • **Mutual Exclusion:** Make all resources sharable (not always possible) • **Hold and Wait:** Require processes to request all resources at once, or release current resources before requesting new ones • **No Preemption:** Allow forced resource preemption (rollback and restart) • **Circular Wait:** Impose ordering on resource types; processes must request resources in increasing order **2. Deadlock Avoidance:** Dynamically analyze resource allocation state to ensure system never enters unsafe state. • **Banker's Algorithm:** Uses maximum demand, current allocation, and available resources to determine if a request can be safely granted. System maintains safe state where a sequence exists for all processes to complete. **3. Deadlock Detection and Recovery:** Allow deadlocks to occur, then detect and recover. • **Detection:** Use wait-for graph (single instance) or resource allocation graph (multiple instances) • **Recovery:** Process termination (kill all or one at a time) or resource preemption (select victim, rollback, starvation prevention) **4. Deadlock Ignorance (Ostrich Algorithm):** Pretend deadlocks never occur. Used in most OS (Windows, Linux) because deadlock prevention is too expensive and deadlocks are rare. If deadlock occurs, reboot the system.

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.